JHANVI PATEL

Ahmedabad, India | jhanvip8507@gmail.com | github | LinkedIn | LeetCode | Codeforces

EDUCATION

Adani University Gujarat, India
Bachelor of Technology in Computer Science and Engineering | CGPA: **8.89/10** 2024 – 2028

TECHNICAL SKILLS

Languages: Go, Java, Python, JavaScript, SQL, C
Competitive Programming: LeetCode: **700+** solved, rating **1816, top 6.93%** | Codeforces: **Specialist**, max rating **1500**
Distributed Systems: CRDT, Redis Pub/Sub, WebSockets, event sourcing, CQRS, horizontal scaling
Concurrency & Systems: Java NIO, goroutines, channels, mutex, atomic ops, circuit breakers, rate limiting
Databases & Storage: PostgreSQL, MongoDB, SQLite, Redis, JDBC, connection pooling
Tools & Infrastructure: Git, Docker, Maven, GitHub Actions CI/CD
CS Fundamentals: Data structures & algorithms, OS, computer networks, OOP
AI & ML: XGBoost, Random Forest, Linear Regression, scikit-learn, NumPy, Pandas

PROJECTS

DocStream — Go · Redis Pub/Sub · WebSockets · PostgreSQL · CRDT · Next.js GitHub ↗ Live ↗

- Built a distributed real-time collaborative editor in Go using a custom **RGA Sequence CRDT** with lexicographic tiebreaking, ensuring convergence across concurrent writers without a central coordinator.
- Designed horizontal scaling via Redis Pub/Sub with instance-UUID echo-loop prevention; synchronized CRDT state across multiple backend nodes with append-only PostgreSQL ops log and periodic snapshotting to minimize cold-start replay cost.
- Engineered token bucket rate limiting, CORS guards, graceful SIGTERM draining, and OpenTelemetry instrumentation across a chi-routed REST and WebSocket API.

GitResolve — Go · go-tree-sitter · SQLite · CI/CD GitHub ↗ Docs ↗

- Built a Git merge-conflict resolution CLI using **go-tree-sitter AST parsing** to classify conflicts into 7 deterministic categories across Go, JS, and TS with specialized per-type resolution strategies.
- Enforced safety-first writes via `os.Root` sandboxing (CWE-22), `flock(2)` advisory locking, atomic file swaps, and pre-write syntax validation with a hard 10MB DoS gate.
- Persisted every conflict decision to SQLite with namespaced reason codes; CI-gateable escalation-rate metrics via `--enforce-gates`; fuzz testing, idempotency tests, and concurrent lock contention validation.

NioFlow — Java 17 · NIO · Multi-threading · Maven Central GitHub ↗ Docs ↗

- Built an HTTP/1.1 framework from scratch using **Java NIO selector loops** and a bounded worker pool with dedicated DB executor isolation, preventing request worker starvation under concurrent JDBC load.
- Achieved **501 RPS** with **99.84% success rate** under 100 VU k6 load test (p50: 1.52ms, p99: 74.62ms); enforced **83.44% instruction coverage** via JaCoCo gate across 271 tests.
- Implemented custom circuit breakers (CLOSED/OPEN/HALF-OPEN) with per-route hedging and distributed Redis rate limiting to prevent cascading failures under load. Added request replay for debugging, OWASP dependency scanning in CI, and a TLS entry point with graceful drainAndStop shutdown.

Vexor — Go · API Gateway · Rate Limiting · Circuit Breaker · Load Balancing GitHub ↗

- Engineered a config-driven Go API gateway with per-route policy composition - rate limiting (token bucket, fixed/sliding window, sliding log) and circuit breakers scoped per route, preventing upstream failure propagation across unrelated routes.
- Implemented **3 load balancing strategies** with health-aware upstream selection; token-bucket limiter benchmarked at **31.7M ops/sec** (31.56 ns/op, 0 allocations/op) on an Intel Ultra 5 processor.
- Hardened reverse proxy strips and rewrites `X-Forwarded-For` from verified socket peer; graceful SIGTERM shutdown with JSON `/metrics` observability endpoint.

EXPERIENCE & LEADERSHIP

DSA Domain Lead — ASPDC, Adani University Aug 2025 – Present

- Taught Sliding Window, Prefix Sum with HashMap, and Greedy patterns to peers; conducted workshops on pattern recognition.

Robotics Trainer — AURC, Adani University Aug 2025-Present

- Taught bitwise operations for sensor and motor state control on Firebird V robots.